GridWatch: how a message travels

Left: the step. Right: the code that does it. Read top to bottom.

Part 1: Kafka (ingest → alert)

Postman
POST /api/readings

→

Receives the HTTP call and returns 202 with the id.

`ingest/controller/ReadingController.java`

```
@PostMapping
public ResponseEntity<Map<String, UUID>> receive(@Valid @RequestBody
UserReadingRequest request) {
    UUID id = readingService.publish(request);
    return ResponseEntity.status(HttpStatus.ACCEPTED).body(Map.of("id", id));
}
```

↓

ingest-service
Kafka producer

→

Sends the reading to the topic. The key is substationId, so the same station always goes to the same partition (keeps the order).

`ingest/service/ReadingService.java`

```
var result = kafka.send(topic, in.substationId(), out).get(5,
TimeUnit.SECONDS);
// Kafka down → KafkaUnavailableException → 503
```

`ingest/application.yaml`

```
producer:
  acks: all                       # wait until Kafka has saved it
  enable.idempotence: true        # no duplicates on retry
```

↓

topic
gridwatch.readings
3 partitions

→

Created at startup. It stores the messages and keeps them after they are read.

`ingest/kafka/Topics.java`

```
@Bean
public NewTopic readingTopic(@Value("${kafka-topic.reading}") String topic) {
    return new NewTopic(topic, 3, (short) 1);   // 3 partitions, 1 copy
}
```

↓

alert-service
Kafka consumer
group alert.gridwatch

→

Reads every reading. The group id remembers the position (offset).

`alert/kafka/Listener.java + application.yaml`

```
@KafkaListener(topics = "${kafka-topic.reading}", groupId =
"${spring.kafka.consumer.group-id}")
public void onReading(ReadingRestRequest r) {
readingService.saveReadingRequest(r); }

consumer:
  group-id: alert.gridwatch
  auto-offset-reset: earliest     # new group starts from the beginning
```

↓

save + rule
reading, work_order

→

Skips duplicates, saves the reading, value above limit → work order.

`alert/service/ReadingService.java`

```
if (r.id() != null && readingRepository.existsById(r.id())) return;   //
duplicate
readingRepository.save(readingEntity);
if (r.value() > sensorTypeOption.limit) saveWorkOrder(readingEntity);
```

↓ if it fails

dead-letter topic
gridwatch.readings-dlt

→

Fails 3 times (1 s apart), or invalid data at once → goes to the DLT.

`alert/kafka/KafkaErrorHandler.java`

```
var errorHandler = new DefaultErrorHandler(recover, new FixedBackOff(1000L,
3));
errorHandler.addNotRetryableExceptions(InvalidReadingException.class);
```

Part 2: RabbitMQ (alert → notifier → email + audit)

alert-service
save work order

→ Saves the work order and raises a Spring event (not yet Rabbit).

`alert/service/ReadingService.java`

```
workOrderPublisher.publishEvent(new WorkOrderEvent(wo.getId(), ...,
WorkOrderType.OPEN));
// closeWorkOrder() does the same with WorkOrderType.CLOSED
```

↓ after the DB commit

publisher
Rabbit publisher

→ Runs only after the commit, so no message is sent for a rolled-back work order.

`alert/rabbit/WorkOrderPublisher.java`

```
@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
public void workOrderCreated(WorkOrderEvent event) {
    rabbit.convertAndSend(exchange, routingKey, event);
}
```

↓ routing key

exchange
gridwatch.work-orders
type: topic

→ The post office. alert only knows this. Declared in both services (same settings = OK).

`alert + notifier /rabbit/RabbitConfig.java`

```
@Bean
public TopicExchange workOrderExchange(@Value("${rabbit.exchange}") String
exchange) {
    return new TopicExchange(exchange);
}
```

↓ bindings decide who gets a copy

**binding work-order.*
queue notifier.work-orders**

```
@Bean Binding notifierBinding(Queue notifierQueue,
        TopicExchange workOrderExchange, ...) {
  return BindingBuilder.bind(notifierQueue)
     .to(workOrderExchange).with(bindingKey);
}
@Bean Queue notifierQueue(...) {
  return QueueBuilder.durable(queueName)
     .deadLetterExchange("")
     .deadLetterRoutingKey(notifierDlq).build();
}
```

↓

WorkOrderListener

```
@RabbitListener(queues = "${rabbit.queue}")
void receiveWorkOrderEvent(WorkOrderEvent e) {
  validateWorkOrderEvent(e);   // bad → no retry
  notificationService.notifyOnce(e);
}
```

↓

NotificationService → Mailpit

```
if (repo.existsById(key)) return;  // sent before
saveNotification(key);
emailService.sendWorkOrder(e);
```

↓ fails 3x / invalid

notifier.work-orders.dlq

**binding work-order.*
queue audit.work-orders**

```
@Bean Binding auditBinding(Queue auditQueue,
        TopicExchange workOrderExchange, ...) {
  return BindingBuilder.bind(auditQueue)
     .to(workOrderExchange).with(bindingKey);
}
@Bean Queue auditQueue(...) {
  return QueueBuilder.durable(queueName)
     .deadLetterExchange("")
     .deadLetterRoutingKey(auditDlq).build();
}
```

↓

AuditListener

```
@RabbitListener(queues = "${rabbit.audit-queue}")
void receiveNotification(WorkOrderEvent e) {
  validateWorkOrderEvent(e);
  auditService.saveLog(e);
}
```

↓

AuditService → log line

```
log.info("LOG event: {}, type: {}, time:{}",
   e.workOrderId(), e.workOrderType(), e.createdAt());
```

↓ fails 3x / invalid

audit.work-orders.dlq

Retry settings (notifier application.yml): `max-attempts: 3, initial-interval: 1s, multiplier: 2`.
`skipRetryForInvalidMessages` in RabbitConfig: an `AmqpRejectAndDontRequeueException` goes to the DLQ at once.

Who owns what

Piece	Owned by	Why
Exchange	sender (alert)	alert only needs to know where to drop the message
Queue + binding + DLQ	receiver (notifier)	each reader decides what it wants; alert never changes
Topic gridwatch.readings	producer (ingest)	created where the data starts
DLT gridwatch.readings-dlt	consumer (alert)	only the reader knows what failed

> **Your sentence, corrected:** alert sends to the **exchange** → each **queue** is bound **to the exchange** with a pattern (not to another queue) → each queue has its **own DLQ** → each queue has **one listener** in a service.

> **Found while drawing:** alert's yaml has one fixed `routing-key: work-order.created`. So a **closed** event is also sent with the key `work-order.created`. It still arrives because `work-order.*` matches everything, but the label is wrong. Your notifier test sends `work-order.closed`, so the test doesn't match real life. Question for you: how would you make the key follow the event type?